

TED UNIVERSITY

SENG-491 HIGH- LEVEL DESIGN REPORT ATTENDAI

Vugar Gurbanov (vugar.gurbanov@tedu.edu.tr) (SENG)

Ahmet Arda Bağdatlı (ahmetarda.bagdatli@tedu.edu.tr) (SENG)

Nuray Aliyeva (nuray.aliyeva@tedu.edu.tr) (SENG)

Contents

1- Introduction	3
1.1- Purpose of The System	3
1.2- Design Goals	3
1.3- Overview.....	3
2- Proposed Software Architecture	4
2.1- Overview	4
2.2- Subsystem Decomposition	4
2.3- Hardware Software Mapping.....	5
2.4- Persistent Data Management	6
2.5- Access Control and Security.....	7
2.6- Global Software Control	8
2.7 Boundary Conditions	8
3- Subsystem Services	9
3.1- App Navigator	9
3.2- Webcam Manager	9
3.3- Database Manager	9
3.4- Embedding Manager	9
4- Referances	10

1- Introduction

1.1- Purpose of The System

AttendAI aims to create a configurable automatic attendance system for teachers to use as they teach to their classes. AttendAI will also allow students to check and easily object to the records.

1.2- Design Goals

DG1- The system must abide the Turkish KVKK regulations.

DG2- The system must be intuitive to navigate through.

DG3- The system must be configurable with a serviceable default.

DG4- The system must delete all images after embedding calculation for privacy.

DG5- The system must encrypt face embeddings before inserting into the database.

DG6- The system must cache students present in a session to allow for faster computing.

DG7- The system must take snapshots in time intervals set in configurations for embedding computing.

1.3- Overview

AttendAI is a project that takes snapshots from a lecture sessions to check which students are active with face embeddings. To achieve this, the system uses both a database and a cache to store embeddings. The system will also store attendance data and allow teachers to edit attendance records while students can review and request changes in the attendance.

2-Proposed Software Architecture

2.1- Overview

AttendAI is planned to be written in Python utilizing both a PostgreSQL for the database and Redis for in RAM cache to speed up queries. To fetch embeddings from faces DeepFace will be used which is a wrapper library that contains many different models. With importance to the KVKK all the embeddings stored on the database will be encrypted while all the images processed will be deleted afterwards.

2.2- Subsystem Decomposition

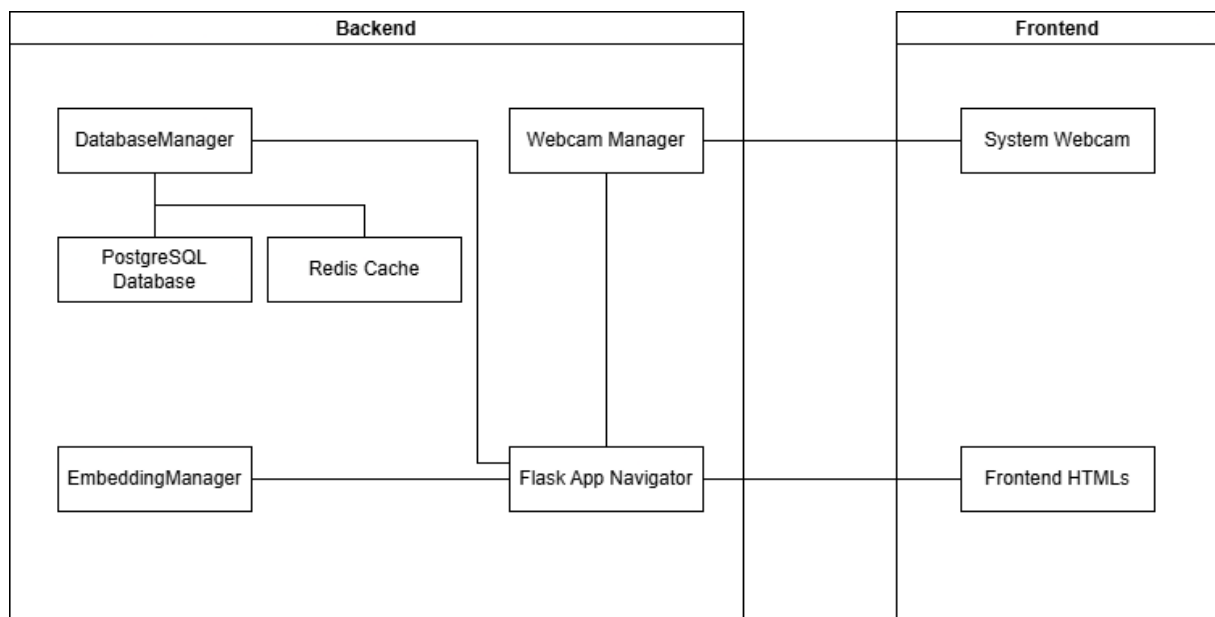
AttendAI will be written to be a website with a backend that contains both the database and the embedding weights. While all the work gets done on the backend the frontend will be more like a display and the camera of the system.

Webcam access -> client-side (browser)

Snapshot transmission -> backend API

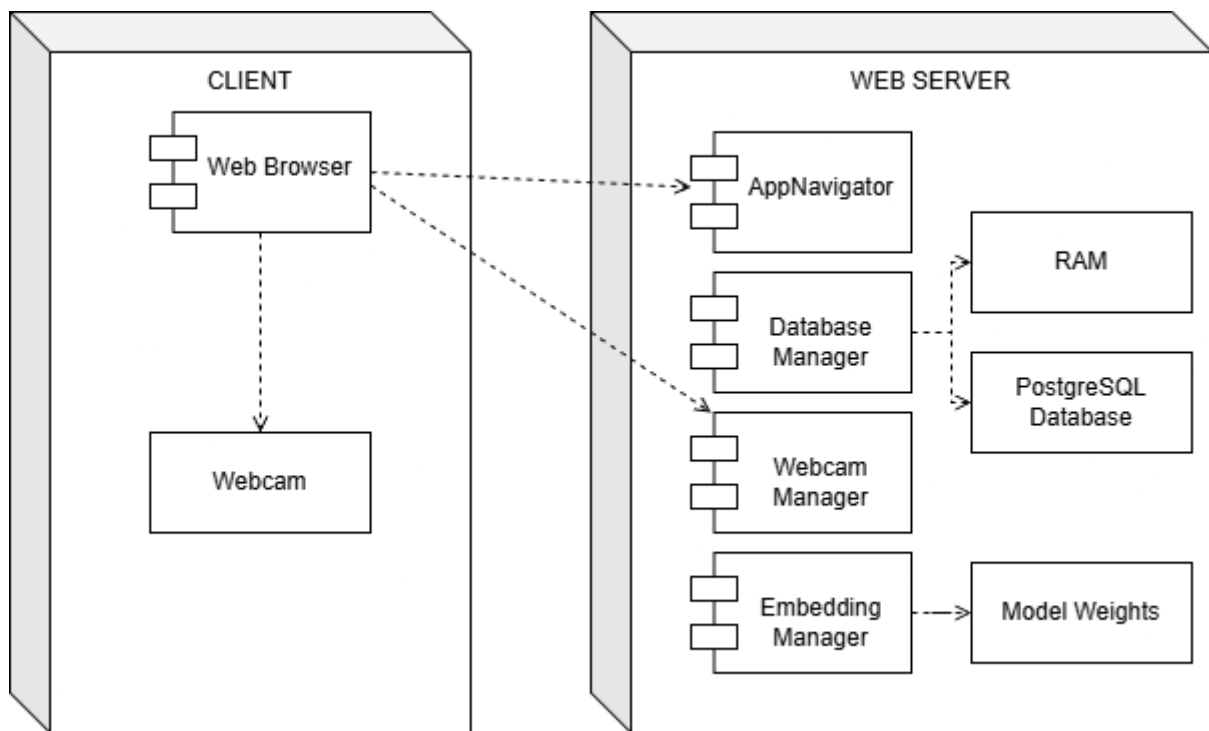
Processing -> backend only

In the backend we have different modules such as a database manager, embedding manager and a webcam manager. While the frontend is managed by the Flask library and HTML files.



2.3- Hardware Software Mapping

The system will be contained in one device that allows for deployment on a server in the future. Specific parts of AttendAI will be mapped to specific parts of the device such as RAM while client-side devices will be used such as the webcam. Additionally model weights for face embedding calculation will be installed on the device and loaded when being used.



2.4- Persistent Data Management

AttendAI will utilize a PostgreSQL database to store data such as user info and attendance data. The database will contain the following tables:

1- Users:

This table contains user ID, login credentials, name and the role of the user.

userID: Primary key. Incrementally generated user ID.

username: Username used in login.

password_hash: Password of the user hashed for security.

role: Assigned role such as admin, student and teacher.

2- Lectures:

This table contains lecture data created by the teacher.

lectureID: Primary key. Incrementally generated lecture ID.

teacherID (Foreign key from Users): User ID of the teacher.

createDate: Creation date of the lecture.

3- FaceEmbeddings:

This table stores face embeddings of the users.

embedID: Primary key. Incrementally generated embed ID.

userID (Foreign key from Users): User ID of the embedding.

embedding: Encrypted embedding generated from the login photo of the user.

model: The model name that generated the embedding.

4- Sessions:

Holds the session data inserted at the end.

sessionID: Primary key. Incrementally generated session ID.

lectureID (Foreign key from Lectures): The lecture ID of the session.

config: An array containing session configuration.

5- Snapshots:

Holds snapshots from all the sessions.

snapshotID: Primary key. Incrementally generated snapshot ID.

sessionID (Foreign key from Sessions): The session ID the snapshots included in.

snapshotDate: The time the snapshot was taken.

6- SnapDatas:

Holds attendance datas for students.

Has the composite key of (snapshotID, userID)

snapshotID (Foreign key from Snapshots): The snapshot ID the data is a part of.

userID (Foreign key from Users): The student the data describes.

attending: Boolean used to signal if the user is attending.

confidence: The similarity of the embedding comparison.

7-LecturesStudents:

Relational table that links students to lectures. Uses a composite key (lectureID, userID)

lectureID (Foreign key from Lectures): The lecture to assign.

userID (Foreign key from Users): The user to assign.

2.5- Access Control and Security

AttendAI will use a role based access system with tokens using the JWT system to carry over access. Our system will use encryption to store embeddings for the privacy of the students and all of our passwords will be hashed before inserting to the database. The system will remove all the images recieved after processing to abide by the KVKK rules as we follow the idea of not saving any images on the system. Tokens will be short-lived and reissued on reauthentication.

The three roles present in the system will be:

System Admin: Has access to an SQL query and can see all the lectures and sessions.

Teacher: This role can create attendance sessions and edit previously made sessions. The teacher role also has access to creating new lectures for future semesters.

Student: The student can view attendance reports involving them and can request a review on their attendance by the teacher.

2.6- Global Software Control

AttendAI will be contained on one singular device with a database and the model weights. This allows us to deploy the software in its own servers for continuous operation. Database connections will be managed through pooled connections to avoid excessive resource usage.

The version control is done using Git.

2.7 Boundary Conditions

Our system has specific boundary conditions that are both ethical and hardware/software oriented. These are:

Legal & Ethical Concerns:

BC1 – The system must comply with Turkish KVKK regulations.

BC2 – The system must not permanently store any facial images.

BC3 – The system must store face embeddings only after encryption.

Hardware Oriented:

BC4 – The system depends on the client device having a functional webcam.

BC5 – Webcam positioning and environmental conditions affect the system's performance.

Software Oriented:

BC6 – The system relies on third-party libraries such as Flask and DeepFace.

BC7 – The system uses an in-memory cache to improve performance; if the cache fails, the system falls back to database operations with reduced performance.

3- Subsystem Services

AttendAI consists of 4 main backend subsystems to function. These are:

3.1- App Navigator

AttendAI will utilise Flask for its frontend website. The app navigator system orchestrates the different HTML pages and the different functionalities of each page in our system.

3.2- Webcam Manager

AttendAI will require the clients webcam in order to take snapshots. This subsystem consists of functionality that requests and manages webcam output such as images in snapshots.

3.3- Database Manager

AttendAI will use both Redis for in memory caching and a PostgreSQL database to store persistent data. The database manager will be used to coordinate SQL queries whenever the app navigator needs data from the database and the database manager will provide cached embeddings whenever requested.

3.4- Embedding Manager

AttendAI uses face embedding models to obtain vectors which will then be compared for similarity. Through this similarity attendance can be monitored. To do this the embedding manager will deal with retrieving the snapshot image and creating a list of embeddings which will then be used for similarity checking.

4- References

- 1- Kişisel Verileri Koruma Kurumu. (n.d.). *KVKK resmî sitesi*. Retrieved November 30, 2025, from <https://www.kvkk.gov.tr/>
- 2- Python Software Foundation. (n.d.). *Python.org*. Retrieved November 30, 2025, from <https://www.python.org/>
- 3- PostgreSQL Global Development Group. (n.d.). *PostgreSQL: The world's most advanced open-source relational database*. Retrieved November 30, 2025, from <https://www.postgresql.org/>
- 4- Pallets Projects. (n.d.). *Flask (A Python micro-framework)*. Retrieved November 30, 2025, from <https://flask.palletsprojects.com/>
- 5- serengil. (n.d.). *DeepFace: A lightweight face recognition and facial attribute analysis library*. Retrieved December 24, 2025, from <https://github.com/serengil/deepface>
- 6- Redis. (n.d.). *Redis: The real-time data platform*. Retrieved December 24, 2025, from <https://redis.io/>